

BIBLE LINUX – \$10 – COMPILATION & DÉVELOPPEMENT

LÉGENDE : [SUDO] = droits root requis | [USER] = utilisateur normal

NOTE : Cette section couvre les outils de base pour compiler, déboguer et profiler le code (C/C++ principalement), ainsi que la gestion de versions avec Git.

A. GESTION DE VERSIONS : GIT

[USER] git

DESCRIPTION : Système de contrôle de versions décentralisé le plus utilisé. Suit l'évolution du code source, facilite le travail en équipe et la gestion de multiples versions (branches).

POURQUOI : Garder l'historique des modifications, collaborer, expérimenter sans risquer de casser le code stable, revenir à une version antérieure en cas de problème.

CONTEXTE : Indispensable pour tout développement moderne.

COMMANDES DE BASE :

git init	Initialise un dépôt dans le répertoire courant
git clone URL	Clone un dépôt distant
git add FICHIER	Ajoute un fichier à l'index (staging)
git commit -m "MESSAGE"	Valide les changements indexés
git status	Affiche l'état des fichiers
git log	Historique des commits
git diff	Différences non indexées
git diff --staged	Différences indexées
git checkout BRANCHE	Change de branche
git checkout -b BRANCHE	Crée et change de branche
git branch	Liste les branches locales
git merge BRANCHE	Fusionne BRANCHE dans la branche courante
git pull	Récupère et fusionne les changements distants
git push	Envoie les commits locaux vers le distant
git remote -v	Liste les dépôts distants
git stash	Met de côté les modifications en cours
git stash pop	Réapplique le stash

EXEMPLES :

```
$ git clone https://github.com/user/projet.git
$ git add mon_fichier.c
$ git commit -m "Ajout de la fonctionnalité X"
$ git push origin main
$ git log --oneline --graph --all # Vue compacte de l'historique
```

B. COMPILATEURS C/C++

[USER] gcc / g++

DESCRIPTION : GNU Compiler Collection – compilateurs pour C (gcc) et C++ (g++). Traduisent le code source en exécutable ou bibliothèque.

POURQUOI : Transformer le code source lisible par l'humain en code machine exécutable par le processeur.

CONTEXTE : Développement en C/C++, compilation de logiciels, noyau Linux.

OPTIONS :

-o FICHIER	Nom du fichier de sortie
-c	Compile mais ne lie pas (génère un .o)
-S	Génère l'assembleur (.s) seulement
-E	Préprocesseur seulement
-g / -g3	Inclut les infos de débogage (niveau 3 = max)
-O0 / -O1 / -O2 / -O3 / -Os	Niveaux d'optimisation
-Wall / -Wextra / -Werror	Active les warnings / tous / erreurs sur warnings
-std=c11 / -std=c++17	Version du langage
-I RÉPERTOIRE	Ajoute un chemin pour les headers
-L RÉPERTOIRE	Ajoute un chemin pour les bibliothèques
-l NOM	Lie avec la bibliothèque libNOM.so
-D MACRO[=VALEUR]	Définit une macro préprocesseur
-U MACRO	Annule une macro
-fPIC	Génère du code indépendant de la position (pour .so)
-shared	Crée une bibliothèque partagée (.so)
-static	Force le linkage statique

```

-pthread           Active le support POSIX threads
-march=native      Optimise pour l'architecture du CPU hôte
EXEMPLES          :
$ gcc -Wall -g -o programme fichier.c           # Compilation standard
$ gcc -c -fPIC fichier.c -o fichier.o           # Pour bibliothèque partagée
$ gcc -shared fichier.o -o libma.so              # Crée la bibliothèque partagée
$ g++ -std=c++17 -O2 -o app *.cpp -lssl -lcrypto # C++ avec bibliothèques externes

```

```
[USER] make
```

```

DESCRIPTION : Automatise la compilation via un fichier Makefile décrivant les
              dépendances et les règles de construction du projet.
POURQUOI    : Éviter de retaper manuellement les longues commandes de compilation,
              ne recompiler que ce qui a changé, gérer des projets complexes.
CONTEXTE    : Outil de build standard pour les projets C/C++.
OPTIONS     :
  -f FICHIER      Utilise FICHIER comme Makefile (défaut: Makefile)
  -j N            Lance N jobs en parallèle (accélère la compilation)
  -k / --keep-going Continue même si erreur sur une cible
  -n / --just-print Simulation (affiche les commandes sans les exécuter)
  -B / --always-make Force la reconstruction de toutes les cibles
  -C RÉPERTOIRE  Exécute make après être allé dans RÉPERTOIRE
  -d / --debug    Mode débogage (affiche les décisions)
  -I RÉPERTOIRE  Ajoute RÉPERTOIRE à la recherche des makefiles inclus
  -s / --silent   Mode silencieux (n'affiche pas les commandes)
  VARIABLE=VALEUR Passe une variable au Makefile
CIBLES COURANTES :
  all             Construit tout (souvent la cible par défaut)
  clean           Supprime les fichiers générés
  install         Installe le programme
  uninstall       Désinstalle
  test            Lance les tests
EXEMPLES        :
$ make            # Construit avec le Makefile par défaut
$ make -j4        # Compilation parallèle (4 jobs)
$ make clean      # Nettoie les fichiers objets
$ make install    # Installe le programme (souvent besoin de sudo)
$ make -n         # Simuler sans exécuter (vérifier avant)

```

C. DÉBOGUEURS & ANALYSEURS

```
[USER] gdb
```

```

DESCRIPTION : GNU Debugger – permet d'exécuter un programme pas à pas, d'inspecter
              la mémoire, les variables, la pile d'appels, et d'analyser les crashes.
POURQUOI    : Trouver l'origine d'un crash, comprendre le comportement d'un
              programme, inspecter l'état mémoire à un instant donné.
CONTEXTE    : Débogage de programmes C/C++, analyse de core dumps.
COMMANDES PRINCIPALES (en interactif) :
  run [ARGUMENTS]  Lance le programme
  break FONCTION / ligne Pose un point d'arrêt
  next / n         Exécute la ligne suivante (sans entrer dans les fonctions)
  step / s         Entre dans la fonction appelée
  continue / c     Continue jusqu'au prochain point d'arrêt
  print EXPR       Affiche la valeur d'une variable/expression
  backtrace / bt   Affiche la pile d'appels
  info locals      Affiche les variables locales
  list            Affiche le code source autour de la ligne courante
  quit            Quitte gdb
EXEMPLES        :
$ gdb ./mon_programme # Lance gdb sur le programme
$ gdb ./mon_programme core # Analyse un core dump
(gdb) break main       # Point d'arrêt sur main
(gdb) run arg1 arg2    # Lance avec arguments
(gdb) bt               # Backtrace après un crash
$ gdb -batch -ex "run" -ex "bt" --args ./prog arg # Non-interactif

```

```
[USER] valgrind
```

```
DESCRIPTION : Suite d'outils pour l'analyse dynamique de programmes. Le plus connu :
```

memcheck (détection de fuites mémoire et erreurs d'accès).

POURQUOI : Détecter les fuites mémoire, les lectures/écritures hors limites, l'utilisation de mémoire non initialisée, les doubles libérations.

CONTEXTE : Débogage mémoire avancé, optimisation, vérification de robustesse.

OUTILS PRINCIPAUX :

memcheck	Détection d'erreurs mémoire (défaut)
cachegrind	Analyse du cache CPU
callgrind	Analyse des appels de fonctions
helgrind	Détection de races dans les programmes threadés
massif	Analyse du tas (heap)

OPTIONS COMMUNES :

--tool=OUTIL	Choisit l'outil (memcheck, cachegrind...)
--leak-check=full	Rapport détaillé sur les fuites
--show-leak-kinds=all	Affiche tous les types de fuites
--track-origins=yes	Trace l'origine des valeurs non initialisées
-v / --verbose	Mode verbeux
--log-file=FICHIER	Écrit la sortie dans un fichier
--suppressions=FICHIER	Ignore certaines erreurs (faux positifs)

EXEMPLES :

\$ valgrind ./mon_programme	# Vérification mémoire simple
\$ valgrind --leak-check=full ./mon_prog	# Rapport détaillé des fuites
\$ valgrind --tool=helgrind ./prog_thread	# Détection de data races
\$ valgrind --tool=callgrind ./prog	# Profilage d'appels
\$ valgrind --suppressions=supp.txt ./prog	# Avec fichier d'ignorés

```
[USER] perf
```

DESCRIPTION : Outil de profilage Linux (Performance Counters) – analyse les performances CPU, les cache-misses, les branches prédites, etc.

POURQUOI : Identifier les goulots d'étranglement CPU, optimiser le code, comprendre l'utilisation des caches et des branches.

CONTEXTE : Optimisation de performances avancée, analyse système.

COMMANDES PRINCIPALES :

stat	Affiche des compteurs de performance
record	Enregistre des échantillons
report	Analyse les échantillons enregistrés
top	Affiche les fonctions les plus coûteuses en direct
list	Liste les événements disponibles

EXEMPLES :

\$ perf stat ./mon_programme	# Stats globales d'exécution
\$ perf record -g ./mon_programme	# Enregistre avec appellations
\$ perf report -g	# Affiche le rapport (fonctions les plus lentes)
\$ perf top	# En direct, ce qui consomme le CPU
\$ perf list	# Liste tous les événements mesurables

D. MANIPULATION DE BINAIRES (COMPLÉMENTS)

```
[USER] ar
```

DESCRIPTION : Crée, modifie et extrait des archives de fichiers objets (.a), typiquement pour construire des bibliothèques statiques.

POURQUOI : Regrouper plusieurs fichiers .o en une seule bibliothèque statique qui pourra être liée à des programmes.

CONTEXTE : Création de bibliothèques statiques, packaging.

OPTIONS :

r	Insère ou remplace des membres
c	Crée l'archive sans avertissement si absente
s	Crée un index (comme ranlib)
t	Liste le contenu de l'archive
x	Extrait les membres
d	Supprime des membres
v	Verbeux

EXEMPLES :

\$ ar rcs libmilib.a fichier1.o fichier2.o	# Crée une bibliothèque statique
\$ ar t libmilib.a	# Liste les fichiers .o contenus
\$ ar x libmilib.a	# Extrait tous les .o

```
[USER] ld
```

DESCRIPTION : Linker GNU (éditeur de liens) – combine des fichiers objets en un

```

exécutable ou une bibliothèque partagée.
POURQUOI : Résoudre les symboles entre les fichiers objets, organiser les
           sections en mémoire, produire le binaire final.
CONTEXTE : Rarement appelé directement (gcc l'appelle en coulisses), utilisé
           pour des linkage complexes (embarqué, scripts de lien personnalisés).
OPTIONS  :
  -o FICHIER      Nom du fichier de sortie
  -e SYM          Point d'entrée (symbole de démarrage)
  -l NOM          Cherche la bibliothèque libNOM
  -L RÉPERTOIRE   Ajoute un chemin de recherche
  -static         Linkage statique
  -shared         Crée une bibliothèque partagée
  -T SCRIPT       Utilise un script de lien personnalisé
  -Map FICHIER    Génère un fichier de mapping
EXEMPLES :
$ ld -o prog fichier1.o fichier2.o -lc      # Linkage basique avec libc
$ ld -shared -o lib.so *.o                 # Crée une bibliothèque partagée
$ ld -T script.ld -o kernel.elf *.o        # Linkage avec script personnalisé

```

```
[USER] as
```

```

DESCRIPTION : Assembleur GNU – traduit le code assembleur (.s) en code objet (.o).
POURQUOI : Quand on écrit directement en assembleur, ou pour générer un .o
           à partir de code généré par un compilateur (gcc -S).
CONTEXTE : Programmation bas niveau, noyau, bootloaders.
OPTIONS  :
  -o FICHIER      Fichier de sortie
  -g              Infos de débogage
  -march=CPU      Architecture cible
  --32 / --64     Mode 32/64 bits
EXEMPLES :
$ as -o fichier.o fichier.s                # Assemble le code source
$ gcc -c fichier.s                         # Alternative (via gcc)

```

```
[USER] nm
```

```

DESCRIPTION : Liste les symboles (fonctions, variables) d'un fichier objet ou
              d'une bibliothèque.
POURQUOI : Voir quels symboles sont définis ou utilisés par un binaire,
           vérifier la présence d'une fonction, diagnostiquer des erreurs de linkage.
CONTEXTE : Analyse de binaires, débogage de linkage.
OPTIONS  :
  -a / --debug-syms  Affiche aussi les symboles de débogage
  -D / --dynamic     Symboles dynamiques (pour les .so)
  -g / --extern-only Symboles externes seulement
  -u / --undefined-only Symboles non définis (importés)
  -C / --demangle    Démangle les noms C++
  -n / --numeric-sort Tri par adresse
EXEMPLES :
$ nm mon_programme      # Tous les symboles
$ nm -C mon_programme   # Avec noms C++ démanglés
$ nm -D /usr/lib/libc.so # Symboles dynamiques de la libc
$ nm -u mon_programme   # Symboles importés (undefined)

```

```
[USER] strip
```

```

DESCRIPTION : Supprime les symboles et informations de débogage d'un binaire
              pour réduire sa taille.
POURQUOI : Alléger les binaires pour la distribution, sans affecter l'exécution.
CONTEXTE : Packaging, préparation de releases.
OPTIONS  :
  -s / --strip-all  Supprime tous les symboles
  -d / --strip-debug Supprime seulement les infos de débogage
  --strip-unnneeded  Supprime les symboles inutiles à la relocation
  -o FICHIER         Fichier de sortie (sinon modifie en place)
EXEMPLES :
$ strip mon_programme      # Supprime les symboles (en place)
$ strip -d mon_programme   # Supprime seulement les infos de debug
$ strip -o programme.mini programme # Vers un nouveau fichier

```

```
[USER] objdump
```

DESCRIPTION : Affiche des informations détaillées sur les fichiers objets :
désassemblage, en-têtes, sections, symboles.

POURQUOI : Désassembler un binaire pour analyse, inspecter les sections,
voir le code généré par le compilateur, reverse engineering.

CONTEXTE : Reverse engineering, analyse de sécurité, optimisation.

OPTIONS :

- d / --disassemble Désassemble les sections exécutables
- D / --disassemble-all Désassemble toutes les sections
- S / --source Intercala le code source (si debug info)
- h / --section-headers Affiche les en-têtes de sections
- t / --syms Affiche la table des symboles
- r / --reloc Affiche les entrées de relocation
- s / --full-contents Affiche le contenu complet des sections
- j SECTION Limite à une section
- M intel Syntaxe Intel (plutôt que AT&T)

EXEMPLES :

```
$ objdump -d mon_programme # Désassemble le code
$ objdump -h mon_programme # Liste les sections
$ objdump -t mon_programme # Table des symboles
$ objdump -d -M intel mon_programme # Désassemble en syntaxe Intel
$ objdump -s -j .rodata mon_programme # Affiche la section .rodata
```

[USER] readelf

DESCRIPTION : Affiche les informations détaillées des fichiers ELF (format exécutable
standard sur Linux).

POURQUOI : Analyser la structure interne d'un binaire ELF : en-têtes, sections,
segments, notes, symboles, etc. Plus fiable que objdump pour l'ELF pur.

CONTEXTE : Analyse de binaires, débogage, compréhension du format ELF.

OPTIONS :

- a / --all Toutes les informations
- h / --file-header En-tête ELF
- l / --program-headers En-têtes de programme (segments)
- S / --section-headers En-têtes de sections
- s / --syms Table des symboles (static)
- dyn-syms Symboles dynamiques
- d / --dynamic Section dynamique
- r / --relocs Relocations
- n / --notes Notes (build-id)
- p SECTION / --string-dump SECTION Dump chaînes d'une section

EXEMPLES :

```
$ readelf -h /usr/bin/bash # En-tête ELF
$ readelf -l /usr/bin/bash # Segments (utile pour l'édition de liens)
$ readelf -s /usr/lib/libc.so # Symboles
$ readelf -d /usr/bin/ssh # Dépendances dynamiques
$ readelf -p .rodata mon_programme # Chaînes de la section .rodata
```

[USER] addr2line

DESCRIPTION : Convertit des adresses mémoire en noms de fichiers et numéros de ligne.

POURQUOI : À partir d'une adresse dans un backtrace (ex: de gdb, d'une erreur),
retrouver la ligne de code source correspondante.

CONTEXTE : Débogage, analyse de crashes (core dumps).

OPTIONS :

- e FICHIER Exécutable contenant les infos de debug
- f Affiche aussi le nom de la fonction
- s Affiche les chemins courts (sans répertoires)
- C Démangle les noms C++
- p Affiche de façon plus lisible
- ADRESSE [...] Adresses à convertir

EXEMPLES :

```
$ addr2line -e mon_programme -f 0x4005a3 # Ligne pour l'adresse
$ gdb ./prog core # Obtenir un backtrace
$ addr2line -e ./prog -f 0x4005a3 0x4006b2 # Plusieurs adresses
$ nm -n mon_programme | grep fonction # Trouver l'adresse d'une fonction
```

[USER] ldd

DESCRIPTION : Affiche les dépendances de bibliothèques partagées d'un exécutable.

POURQUOI : Vérifier quelles bibliothèques .so sont requises et si elles sont
trouvées par le chargeur dynamique.

CONTEXTE : Dépannage d'erreurs "library not found", vérification de linkage.
 EXEMPLES :
 \$ ldd /usr/bin/ls # Bibliothèques utilisées par ls
 \$ ldd -v /usr/bin/ssh # Version verbeuse
 \$ ldd -u /usr/bin/programme # Affiche les dépendances inutiles
 \$ LD_LIBRARY_PATH=/mon/chemin ldd ./prog # Test avec un chemin personnalisé

[USER] ctags

DESCRIPTION : Génère un fichier index (tags) pour naviguer rapidement dans le code source (sauts vers définition de fonctions/variables).
 POURQUOI : Accélérer la navigation dans de grands projets (Vim, Emacs, etc.).
 CONTEXTE : Édition de code, lecture de code source.
 OPTIONS :
 -R Parcours récursif des répertoires
 -h EXT Extensions de fichiers à traiter comme headers
 --c++-kinds=... Types de symboles C++ à indexer
 --fields=... Champs à inclure
 -f FICHIER Fichier de sortie (défaut: tags)
 EXEMPLES :
 \$ ctags -R . # Indexe tout le projet
 \$ ctags -R --c++-kinds=p --fields=+iaS . # Indexation plus complète pour C++
 \$ vi -t fonction # Ouvre Vim directement sur la définition

E. LANGAGES DE SCRIPT (COMPLÉMENTS)

[USER] python3 / pip

DESCRIPTION : Python 3 – interpréteur pour ce langage de script polyvalent.
 pip – gestionnaire de paquets pour installer des bibliothèques Python.
 POURQUOI : Python est omniprésent pour les scripts, l'automatisation, l'analyse de données, le web, etc.
 CONTEXTE : Scripting, développement web, data science, automatisation système.
 COMMANDES PIP PRINCIPALES :
 pip install NOM Installe un paquet
 pip install --user NOM Installe pour l'utilisateur seulement
 pip uninstall NOM Désinstalle
 pip list Liste les paquets installés
 pip freeze Liste les paquets avec versions (pour requirements.txt)
 pip search MOT Recherche des paquets
 pip show NOM Affiche les détails d'un paquet
 pip install -r requirements.txt Installe depuis un fichier
 EXEMPLES :
 \$ python3 script.py # Lance un script
 \$ python3 -m venv .venv # Crée un environnement virtuel
 \$ pip install numpy pandas matplotlib # Installe des libs scientifiques
 \$ pip freeze > requirements.txt # Sauvegarde les dépendances
 \$ pip install -r requirements.txt # Restaure les dépendances

[USER] node / npm

DESCRIPTION : Node.js – runtime JavaScript côté serveur.
 npm – Node Package Manager, gestionnaire de paquets.
 POURQUOI : Développer des applications JavaScript/TypeScript, outils front-end.
 CONTEXTE : Développement web, outils de build (webpack, vite), API.
 COMMANDES NPM PRINCIPALES :
 npm init Initialise un projet (crée package.json)
 npm install / npm i Installe les dépendances
 npm install NOM Installe un paquet
 npm install --save-dev NOM Installe en dépendance de développement
 npm install -g NOM Installe globalement
 npm uninstall NOM Désinstalle
 npm update Met à jour les dépendances
 npm run SCRIPT Exécute un script défini dans package.json
 npm list Liste les paquets installés
 EXEMPLES :
 \$ node -e "console.log('Hello')" # Exécute du code direct
 \$ node mon_script.js # Lance un script
 \$ npm init -y # Initialise projet avec réponses par défaut
 \$ npm install express # Installe Express

```
$ npm install -g nodemon  
$ npm run dev
```

```
# Installe nodemon globalement  
# Lance le script "dev" du package.json
```

F. ANALYSE ET RÉTRO-INGÉNIERIE

[USER] cutter-re / rizin

DESCRIPTION : Cutter est une interface graphique pour le framework Rizin (ex-radare2).
Rizin est un toolkit de rétro-ingénierie et d'analyse de binaires.

POURQUOI : Analyser des binaires sans code source, reverse engineering,
comprendre le fonctionnement d'un malware ou d'un logiciel propriétaire.

CONTEXTE : Sécurité, analyse de malware, rétro-ingénierie.

FONCTIONNALITÉS :

- Désassemblage interactif
- Analyse de graphe de flot de contrôle
- Recherche de chaînes / motifs
- Patch de binaires
- Débogage intégré
- Support multi-architectures

EXEMPLES :

\$ rizin /usr/bin/ls	# Analyse en ligne de commande
\$ cutter	# Lance l'interface graphique
# Dans rizin :	
> aaaa	# Analyse automatique
> s main	# Va à main
> pdf	# Affiche la fonction désassemblée
> VV	# Vue graphe
